

Parallel Programming For Machine Learning Project

Angelo Rosa, Roberto Magrini

Settembre 2026

1 Introduction

The following document is an educational project made for the course *Parallel Programming For Machine Learning*. Although the title suggests that the arguments involved do not threaten any Machine Learning algorithm, it underlines the difficulty and differences in parallel implementations.

The authors of the document do not assume any responsibility for illegal uses of the provided software, especially for the LM Hash Cracker.

The LAN Manager is a protocol created by Microsoft and used in the 80s, initially included in the **SMB** protocol. However, it still appears in many Active Directory protocols such as LDAP and Kerberos, for legacy and older-standard compatibility reasons. If some years ago this was not a real problem, today it is. In fact, with modern hardware, the old algorithm underlines some weaknesses that can be successfully exploited to recover up to a 14-byte-long password in a bunch of minutes, depending on the hardware. Full weaknesses and algorithm will be discussed later. Fundamentally, LAN Manager relies on the LM hash, a cryptographic function designed to store and authenticate user passwords. While Microsoft eventually introduced the more secure NT LAN Manager (NTLM) and later adopted Kerberos as the primary authentication protocol, the LM hash was retained for decades to ensure backward compatibility with older clients like Windows 95 and Windows ME. Although modern Windows operating systems; starting with Windows Vista and Server 2008; disable LM hash storage by default, misconfigured legacy environments or specific backward-compatibility requirements often inadvertently leave this attack surface open. LM hash cracker implemented is a **pure brute-forcer** program which scans the entire space-set of all combinations possible, it does not require any external wordlist or any other input apart from the hash itself.

On the other hand, we also implemented the Bloom Filter as the second project argument. The Bloom filter has completely different challenges, which involve more memory-bound operations than CPU-bound operations. Bloom Filters are space-efficient data structures that are primarily used as a mean to check if an element is part of a set. Since it is space-efficient, it is usually used with a great volume of data (e.g during a sign in it is used to check if the given password is acceptable or not). Generally, a Bloom Filter is an array of bits (could also be a boolean or byte array) initialized to 0. To insert an element, the element first needs to be converted into indices through the use of **k** hash functions, returning **k** indices. Once converted, the positions in the bit array indicated by the indices are set to 1 (this will lose all information about the element). The check works in the same way: the element to be searched is converted into indices, and then the Bloom Filter checks whether the value stored in those positions is 1 or 0. If all positions contain 1s, the element is part of the set; otherwise, if even one 0 is present, the element is not part of the set.

We want to underline that although the 2 project arguments were reviewed by the other partner, we organized our work by splitting the tasks in:

- **BloomFilter** *implemented by Roberto*
- **LM Hash Cracker** *implemented by Angelo*

The project source code and compilation directives for each can be found at GitHub Repository

2 LM Hash Cracker

2.1 Compilers, Framework and Hardware Used

Before going on it is important to underline the hardware and compilers used during the benchmarks.

```
nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005–2026 NVIDIA Corporation
Built on Fri_Apr_24_07:22:02_PM_PDT_2026
Cuda compilation tools, release 13.3, V13.3.33
Build cuda_13.3.r13.3/compiler.37862127_0
```

```
gcc --version
gcc (GCC) 16.1.1 20260430
Copyright (C) 2026 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

To compile you can just run *make* from terminal and you'll find 2 binaries generated:

- `lm_hash_cuda`
- `lm_hash_openmp`

Hardware

- **CPU:** Intel Core i5-12600K (10 Physical Cores: 6 Performance-cores with Hyper-Threading + 4 Efficient-cores, for a total of 16 Threads [12 managed by P-Cores via Hyper-Threading + 4 managed by E-Core])
- **GPU:** NVIDIA GeForce RTX 3060 Ti (Ampere Architecture, 4,864 CUDA Cores)
- **RAM:** 32 GB DDR4 @ 3000 MHz

2.2 Algorithm and Challenges Meet

The LM Hash algorithm is not difficult to understand, but still difficult to implement especially in its CUDA version, since the hash requires the official *DES NIST fips 46-3* implementation <https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf>, which has been **fully re-implemented from scratch** since nothing about this version in CUDA has been found online already working. Trying to use OpenCL kernels by hashcat, required more adaptation than expected so the decision was to proceed for a full re-implementation.

The algorithm proceeds as follows:

1. The user's password is restricted to a maximum of fourteen characters
2. The user's password is converted to uppercase, this reduces the total combinations to test from 95^7 to 69^7 , a big degradation.
3. The user's password is encoded in the System OEM code page. This step has actually been ignored because we will work with ASCII characters only.
4. The password is NULL-padded to 14 bytes
5. The "fixed-length" password is split into two 7-byte halves. This is the real problem of this standard. Even with 69 characters, with a full 14 characters bruteforce the total amount of combinations would be 69^{14} , which even with an obsolete algorithm like DES behind, would be computationally infeasible for consumer hardware in a reasonable time.

6. These values are used to create two DES keys, one from each 7-byte half, by converting the seven bytes into a bit stream with the most significant bit first, and inserting a parity bit after every seven bits (so 1010100 becomes 10101000). This generates the 64 bits needed for a DES key. (A DES key ostensibly consists of 64 bits; however, only 56 of these are actually used by the algorithm. The parity bits added in this step are later discarded.)
7. Each of the two keys is used to DES-encrypt the constant ASCII string **KGS!@#\$\$%** resulting in two 8-byte ciphertext values. The DES CipherMode should be set to ECB, and PaddingMode should be set to NONE.
8. These two ciphertext values are concatenated to form a 16-byte value, which is the LM hash.

The proposed charset is composed of all printable characters and mostly matches those used as the full charset `?a` in hashcat, excluding the lowercase charset as they are converted by the algorithm in uppercase.

```
char FULL_CHARSET[] = "ABCDEFGHJKLMNPQRSTUVWXYZ0123456789!\",#$% \
\"%&'()*+,-./:;<=>?@[\\]^_`{|}~";
```

2.3 Search Space Calculation

For a representative password of length $L = 14$ (e.g. **TEST123TEST123**), the nominal total number of candidate passwords is:

$$C(14) = 2 \times 69^7 = 14,892,706,505,178 \approx 1.489 \times 10^{13} \quad (\approx 14.9 \text{ trillion candidates}) \quad (1)$$

2.4 Peak GPU Throughput

We derive the peak throughput H_{peak} of our custom CUDA implementation from the $L = 7$ measurement of **TEST123** ($T = 37.00 \text{ min} = 2,220 \text{ s}$, $C(7) = 69^7 = 7,446,353,252,589$):

$$H_{\text{peak}} = \frac{C(7)}{T} = \frac{7,446,353,252,589}{2,220} \approx 3.354 \times 10^9 \text{ hashes/second} \quad (\approx 3.35 \text{ GH/s}) \quad (2)$$

This value matches the throughput we independently observe across all rows of Table 1 ($L = 7\text{--}13$, $\approx 3.35 \text{ GH/s}$). All of these measurements are taken over relatively short executions (well under an hour), and are therefore unaffected by sustained GPU thermal throttling; we adopt H_{peak} as the device’s short-run peak rate.

2.5 True Worst-Case Search Space and Sustained Runtime

Because `indexToCandidate` enumerates the charset in a fixed order, the genuine worst case is not merely the longest password, but the one whose candidate index is enumerated *last*. Since the space character (`0x20`) occupies the final position (index 68) of `FULL_CHARSET`, a 14-character password composed entirely of spaces forces an exhaustive search over *all* lengths $L = 1, \dots, 7$ for *both* LM halves:

$$C_{\text{worst}}(14) = 2 \sum_{L=1}^7 69^L = 15,111,716,894,958 \approx 1.511 \times 10^{13} \text{ candidates} \quad (3)$$

We measured this worst-case password twice, under identical code (S-boxes kept in global `__device__` memory in both cases, as throughout the rest of the implementation), differing only in whether the two LM halves were processed concurrently on two CUDA streams or sequentially on a single stream:

- **Concurrent (two CUDA streams):** $T_{\text{streamed}} = 15,549.4711 \text{ s} \approx 4.319 \text{ hours}$
- **Sequential (halves processed one after another):** $T_{\text{sequential}} = 14,914.6760 \text{ s} \approx 4.143 \text{ hours}$

The sequential version without stream can be found on the branch: `lm-hash-cracker-without-stream-usage`. The two runtimes differ by only 4.08%, ruling out significant inter-stream resource contention. Since the concurrent implementation is architecturally simpler and shows no measurable penalty, we retain it in our final implementation and adopt its measurement as the reference worst-case runtime:

$$T_{\text{GPU_custom}} = 15,549.4711 \text{ s} \approx 4.319 \text{ hours} \quad (4)$$

From this measurement we obtain the sustained throughput:

$$H_{\text{sustained}} = \frac{C_{\text{worst}}(14)}{T_{\text{GPU_custom}}} = \frac{15,111,716,894,958}{15,549.4711} \approx 9.72 \times 10^8 \text{ hashes/second} \quad (\approx 0.97 \text{ GH/s}) \quad (5)$$

We attribute the difference between H_{peak} ($\approx 3.35 \text{ GH/s}$, short runs) and $H_{\text{sustained}}$ ($\approx 0.97 \text{ GH/s}$, the multi-hour worst-case run) to GPU boost-clock throttling under prolonged thermal and power load: the worst-case run keeps the device at 100% utilization for over four continuous hours, versus well under an hour for every other measurement in Table 1, consistent with typical consumer GPU power-management behavior. Thermals reached 78° degrees in the long run which is pretty high considering the used device architecture.

As supporting evidence, we compare the representative case (TEST123TEST123, total work 6.816×10^{12} candidates, $T \approx 4,386\text{--}4,439 \text{ s}$) against the worst case (total work 1.511×10^{13} candidates, $T = 15,549.4711 \text{ s}$). The workload increases by $2.22\times$, while the runtime increases by $3.55\times$. The exhaustive length 1–6 phase, common to both, takes nearly identical time in each case (224.86 s vs. 228.23 s , a 1.5% difference), which localizes the additional slowdown to the length-7 phase; precisely the portion of the run long enough for thermal throttling to become measurable.

2.6 Throughput Metrics

Using the sustained worst-case throughput and its associated CPU projection:

- **Custom CUDA Implementation:**

$$\text{Throughput}_{\text{GPU_custom}} \approx 9.72 \times 10^8 \text{ hashes/second} \quad (\approx 0.97 \text{ GH/s}) \quad (6)$$

- **OpenMP (CPU) Implementation (projected):** Using the empirically measured CPU throughput of 56.08 MH/s (Table 1) applied to the same worst-case search space $C_{\text{worst}}(14)$:

$$T_{\text{OpenMP}} = \frac{15,111,716,894,958}{56.08 \times 10^6} \approx 269,467.1 \text{ s} \approx 3.119 \text{ days} \quad (7)$$

$$\text{Throughput}_{\text{OpenMP}} \approx 5.608 \times 10^7 \text{ hashes/second} \quad (\approx 56.08 \text{ MH/s}) \quad (8)$$

- **Hashcat (Reference):** Provided for context only and excluded from the speedup comparison below, since we benchmarked Hashcat independently rather than running it to completion on the same search space. Empirically measured throughput on the same hardware: $\text{Throughput}_{\text{Hashcat}} \approx 30 \text{ GH/s}$, roughly $31\times$ our sustained worst-case throughput, thanks to advanced kernel-level optimizations (bit-slicing and advances table storage and management) that fall outside the scope of this project.

2.7 Speedup and Time Saved

Comparing our custom CUDA implementation against its OpenMP (CPU) counterpart on the true worst-case search space, using $S = \frac{T_{\text{OpenMP}}}{T_{\text{GPU_custom}}}$:

$$S = \frac{269,467.1 \text{ s}}{15,549.4711 \text{ s}} \approx 17.33\times \quad (9)$$

The percentage of time saved by our GPU implementation over the CPU implementation, $E = \max(0, (1 - \frac{1}{S}) \times 100)$:

$$E = \left(1 - \frac{1}{17.33}\right) \times 100 \approx 94.23\% \quad (10)$$

So, the **CUDA Time** column is empirically measured, while for the **OpenMP Time** our empirical tests end at password TEST123 (length 7) because longer runs would be computationally prohibitive; all remaining OpenMP figures are extrapolated from the empirically measured CPU throughput

Table 1: Performance Comparison between CUDA and OpenMP Implementations

N	Plaintext	Hash	CUDA Time	OpenMP Time	GPU Throughput	CPU Throughput	Speedup	Time Saved (%)
1	T	417EAF50CFAC29C3AAD3B435B51404EE	0.0108 s	0.0010 s	6.39 KHash/s	69.0 KHash/s	0.0926	0.0%
2	TE	CEC18980D4FFADA7AAD3B435B51404EE	0.0116 s	0.0009 s	0.410 MH/s	5.29 MH/s	0.0776	0.0%
3	TES	F726D4121A092D9AAD3B435B51404EE	0.0112 s	0.0116 s	29.33 MH/s	28.32 MH/s	1.0357	3.45%
4	TEST	01FC5A6BE7BC6929AAD3B435B51404EE	0.0227 s	0.4927 s	0.999 GH/s	46.01 MH/s	21.70	95.39%
5	TEST1	E88D94D6EBD10FC7AAD3B435B51404EE	0.6309 s	28.16 s	2.479 GH/s	55.54 MH/s	44.64	97.76%
6	TEST12	50081C6A6EDD109BAAD3B435B51404EE	31.77 s	32.07 min	3.397 GH/s	56.08 MH/s	60.58	98.35%
7	TEST123	624AAC413795CDC1AAD3B435B51404EE	37.00 min	1.536 days (est.)	3.354 GH/s	56.08 MH/s	59.81	98.33%
8	TEST123T	624AAC413795CDC1417EAF50CFAC29C3	37.00 min	1.536 days	~3.35 GH/s	~56.08 MH/s	~59.81	~98.33%
9	TEST123TE	624AAC413795CDC1CEC18980D4FFADA7	37.00 min	1.536 days	~3.35 GH/s	~56.08 MH/s	~59.81	~98.33%
10	TEST123TES	624AAC413795CDC1F726D4121A092D9A	37.00 min	1.536 days	~3.35 GH/s	~56.08 MH/s	~59.81	~98.33%
11	TEST123TEST	624AAC413795CDC101FC5A6BE7BC6929	37.00 min	1.536 days	~3.35 GH/s	~56.08 MH/s	~59.81	~98.33%
12	TEST123TEST1	624AAC413795CDC1E88D94D6EBD10FC7	37.01 min	1.537 days	~3.35 GH/s	~56.08 MH/s	~59.81	~98.33%
13	TEST123TEST12	624AAC413795CDC150081C6A6EDD109B	37.53 min	1.559 days	~3.35 GH/s	~56.08 MH/s	~59.82	~98.33%
14	TEST123TEST123	624AAC413795CDC1624AAC413795CDC1	1.233 h	3.073 days	~3.35 GH/s	~56.08 MH/s	~59.81	~98.33%

(56.08 MH/s). The **GPU Throughput** values shown here (rows 4–14) reflect the peak, throttling-free rate $H_{\text{peak}} \approx 3.35$ GH/s that we measure over short executions; Section 2.5 reports the sustained rate $H_{\text{sustained}} \approx 0.97$ GH/s that we observe over the multi-hour worst-case run, where GPU thermal throttling becomes measurable.

2.8 CUDA GPU Version - in-depth technical analysis

As anticipated, the implementation in **CUDA** of the algorithm is not that straightforward, so for more complex technical parts (for example, eventually explaining the **DES** implementation details) we rely on specific questions during the oral interview. Here, instead, it will follow in a more general way how the code is organized and the rationale behind the decisions made.

The algorithm firstly performs an optimization checks, (please note that these checks are also performed by the OpenMP version): since we’re assuming to crack password and not raw bytes, it is assumed to be impossible to have the second 7 bytes of the password not empty if the first 7 bytes are all null bytes (**0x00**). Then, another optimization is performed which checks, with higher priority, if the second half of the password is composed by only null bytes, if yes, it is assumed that the password is less or equal than 7 character, so the flag to skip the computation of the second half of the password is set to 1, and the stream initialization is completely skipped.

Threads number per block are dynamically asked to the GPU using the function **cudaOccupancyMaxPotentialBlockSize** and here the first real bug solved. For passwords 7 bytes long, the algorithm at the end used to print out empty passwords. This was a pretty strange behavior and clearly a bug. The fact is that the bug was caused by the **MaxGrid dimension** (2147483647 in our case). In fact with 7 characters, the total computation for a password exceeded the max grid dimension of the GPU, resulting in a silent failure from the cuda streams. The solution here was to divide the number of total passwords (69^7), assigning to each cuda stream the total combinations of passwords if the number of combinations is less than the chunkSize dimension (which corresponds to the MaxGrid dimension multiplied by the number of threads per block); otherwise we calculate the number of chunks for splitting the work and we pass the maximum assignable amount of work (which is equals to chunkSize) for each chunk.

As someone might object, by looking at the code, it does not make too much sense to use 2 streams, assigning each one to crack half of a password. This is because, when running a single stream, the function **crackHalfKernel** already uses 100% of the cores of the GPU. This means that having 2 streams running and computing the same size will double the required time to crack the hash. This is true especially because streams are helpful for in-memory or not so expensive ALU bounded operations, but here, our algorithm just relies on math tasks, which do not require particular scheduling overhead. The real reason why the cracking was implemented using 2 streams is that this is more elegant in the code, at least in our opinion. After all, the alternative would have to iteratively proceed on the first half of the hash and then, when the first one has been cracked, eventually proceed on the second half of the password (you can actually still find this implementation in the branch lm-hash-cracker-without-stream-usage). This is practically the same amount of time and computation; assigning the work to the 2 streams results in not only a more elegant solution, but also relies entirely on the GPU scheduling and in-memory optimizations for the few bytes to swap in and out from threads’ registers. Not only this, assuming that the first half is, for example, 7 bytes long and the second part is 5 bytes long, with 2 streams working parallelly, the second half is anyway cracked in few seconds thus, once the second

half is cracked, threads and resources are automatically released and re-assigned for the first stream, which since by our assumption is computing a 7 bytes long password, it is surely still processing. So, in summary, the usage of the streams is not only more elegant but actually useful whenever the two 7 bytes halves of the password have different lengths among each other.

The remaining interesting function that may need attention is how thread-to-candidate mapping works. The function which performs such operation is *indexToCandidate* implemented as follows:

```
--device-- void indexToCandidate(uint64_t index, int len, char* out_str) {
    // Da destra a sinistra
    for (int i = len - 1; i >= 0; i--) {
        // Carattere in base alla posizione corrente del thread
        out_str[i] = FULL_CHARSET[index % CHARSET_SIZE];
        index /= CHARSET_SIZE; // Avanza alla posizione successiva
    }
    out_str[len] = '\0'; // terminatore stringa, restituisci il candidato
}

--global-- void crackHalfKernel(
    const uint8_t target_bytes[8],
    char* cracked_out,
    uint64_t start_index,
    uint64_t total_work,
    unsigned int candidate_len,
    unsigned int* found_flag
) {
    // Calcolo dell'ID globale del thread
    // blockDim.x = 768 come suggerito da cudaOccupancyMaxPotentialBlockSize
    // perche' ogni blocco composto da 768 threads
    // blockIdx.x = id del blocco
    // threadIdx.x = posizione generica di un dato thread nel blocco
    uint64_t gid = blockIdx.x * (uint64_t)blockDim.x + threadIdx.x;
    uint64_t idx = start_index + gid; // rappresenta l'id reale e univoco del thread

    // Senza idx con password da 7 byte si ha un bug:
    // la dimensione della griglia eccede il max.
    // Soluzione: usare chunks.
    if (idx >= total_work || *found_flag != 0) {
        return; // Guard closure
    }

    // Calcola il candidate. Allocazione fissa a 8 byte
    // per evitare allocazioni dinamiche sullo stack.
    char candidate[8];
    indexToCandidate(idx, candidate_len, candidate);

    // Encrypt candidate
    uint8_t candidate_block[8];
    hash_half_fast(candidate, candidate_len, candidate_block);

    // Check against known target
    if (compareEqualBytes(candidate_block, target_bytes)) {
        if (atomicExch(found_flag, 1u) == 0) {
            // Copia il candidato trovato nel buffer di output
            for (unsigned int i = 0; i < candidate_len; i++) {
                cracked_out[i] = candidate[i];
            }
            cracked_out[candidate_len] = '\0';
        }
    }
}
```

```

    return ;
}
}

```

This is the core of the real difference between the CPU version (OpenMP) and the CUDA GPU version. The candidate is calculated dynamically by processing the index thread number and mapping it numerically to the candidate thanks to the fixed charset size. The **atomicExch** guarantees that only 1 thread will copy the found candidate into the output buffer. The **start** position is calculated based on the current chunk assigned to the kernel, over the total calculated in the main loop. The standard computation of **gid** and **idx**, which can be found in most CUDA kernels (1d size), gives spatial information to find the specific thread to assign the computation; it maps the grid into one single continuous array.

One last observation to make regarding this part is the treatment of the DES permutation tables. In fact, in the **des.cu** it can be seen that all permutation tables are stored in the constant memory using the **__constant__** directive, but storing the **S_BOX** lookup table in the same way, so in the constant memory, resulted in drastically reducing the performance of the cracking speed, but not only this; in fact, with this minor change, the usage of the GPU cores rarely reaches 100%, even with a full intensive cracking task on a 7-byte half. We actually don't know the real reason behind this, but we can hypothesize that one reason is that the S_box returns different index results for each password guess. Since each password guess is performed by a different thread, the attempt to access a smaller memory like **__constant__** is fully serialized in atomic operations, since its large dimension exceeds the optimal cache line size. So the problem could be the fact that the large number of threads are trying to access the same memory portion at the same time simultaneously.

2.9 OpenMP CPU Version - in-depth technical analysis

Regarding the OpenMP version, technically there is not that much to say; the most interesting part was the parallel implementation of the algorithm. In fact, to create the parallel version, we first created the iterative normal one on the CPU to better study the algorithm. After that, the only changes made were adding pragma directives to parallelize the operations. Notice the parallelized code is practically the same as the CUDA version, allowing a real one-to-one comparison between the two.

In fact, via the OpenMP **section** directives, the two halves of the hash are split and manipulated concurrently. Actually, in OpenMP, there is also another OpenMP parallelisation which is performed on different lengths. As can be noticed, the scheduling of the threads is dynamic, leaving OpenMP in auto mode regarding the thread scheduling directives and management. Although it is the simplest implementation, the speed of $56.08MH/s$ is not so irrelevant for a CPU. Regarding the optimizations, they're the same as the already discussed null-byte checks optimizations performed by the CUDA version. There is only one method which is really different and is useful to underline; this is **build_combo_dfs**. The method is interesting because the CUDA version, as already explained, uses a map based on thread indexing for calculating the next candidate. Here instead, which version was written actually before the CUDA implementation, uses a *Depth-First Search* algorithm for calculating the next candidate, since calculating 69^7 of candidates not only would take probably more of the time of the cracking itself, but would eventually lead the dynamic or the stacked memory to explode due to the huge dimensions of the full combinations. Here follows some maths that better underlines this concept:

Pre-generated Candidate List

$$\begin{aligned}\mathcal{T}_{\text{list}} &= \mathcal{O}(N^L) \\ \mathcal{S}_{\text{list}} &= \mathcal{O}(L \cdot N^L)\end{aligned}$$

Standard iterative approach that pre-generates and stores all possible candidates in memory before evaluation.

- $N = |\Sigma|$ (alphabet size, e.g., 69)
- $L = \text{maximum password length (7)}$

Depth-First Search (DFS)

$$\mathcal{T}_{\text{dfs}} = \mathcal{O}(N^L)$$

$$\mathcal{S}_{\text{dfs}} = \mathcal{O}(L)$$

Recursive traversal that generates and tests candidates on-the-fly, reclaiming memory via backtracking.

Key Differences

- **Time Complexity ($\mathcal{T}$):** Identical ($\mathcal{O}(N^L)$). Both methods evaluate the exact same exponential number of total combinations ($K = \sum_{k=1}^L N^k$).
- **Space Complexity ($\mathcal{S}$):** The pre-generated list requires **exponential space** ($\mathcal{O}(L \cdot N^L) \approx 61$ TB of RAM for $L = 7$), making it unfeasible. DFS reduces this to **linear space** ($\mathcal{O}(L)$, a few bytes on the stack).

Another important difference is the use of OpenSSL to encrypt the candidate with the DES algorithm. In fact, if in CUDA the **des.cu** represents a real re-implementation of the DES, instead here, we use the OpenSSL library which actually should not be a problem, the opposite instead! In fact, it is uncommon to rewrite cryptographic algorithms for security and performance reasons, so given that this is a dedicated library whose purpose is to implement standard crypto algorithms, we can be quite sure it is better than any custom DES implementation. The **des_encrypt_block** is the function which is used to encrypt the current candidate block, and it can be seen that it initializes and frees the OpenSSL directives each time is called. This is the only overhead included in the code, in contrast to the GPU version, but since using a single shared context could lead to undefined behavior, we decided to accept this small difference.

3 Bloom Filter Parallelized

3.1 Compilers, Framework and Hardware Used

The testing iterations of both implementations have been conducted on the same machine, with the following specifications of interest:

- **CPU:** AMD Ryzen 7 3750H (8 physical threads) with x86-64 architecture.
- **RAM:** 2 x 8GiB SODIMM DDR4 Synchronous Unbuffered 2667 MHz
- **CACHES:** 384KiB L1 cache, 2MiB L2 cache, 4MiB L3 cache.

The following compilers have been used in this work:

- **C++ Compiler:** `g++ -version` — `g++` (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0 Copyright (C) 2023 Free Software Foundation, Inc.
- **Python Version:** Python 3.13.15 GIL free

The frameworks, libraries and modules utilized are:

- **OpenMP**, with pragma directives.
- **MurmurHash3**, used to optimize hashes calculations.
- The module **ThreadPool** from the library multiprocessing, to achieve true multithreading on Python.

3.2 Algorithm and Challenges Meet

The objective of this work is to understand how much improvement is achieved using parallelization techniques and how much it differs between programming languages. For this work, two programming languages have been analyzed: C++ and Python, with OpenMP and pragma directives for the C++ implementation and with a Python free-threaded version (an experimental version that allows deactivating the GIL and achieving real multithreading via thread pooling) for the second implementation. The challenges faced in this work are, firstly to understand how parallelization works in this two different environments, secondly, to understand what parts of the Bloom Filter algorithm are parallelizable, and lastly, to gather the data needed to make an educated consideration on the parallel implementations.

To compare sequential and parallel performances, both implementations are divided into two parts: in one part of the implementations, the operations and algorithms are done sequentially, while in the other part, the operations use parallelisation techniques. Also, to correctly compare the two implementations, both of them must follow the same strategies used for parallelisation (for example, if one implementation utilises a shared byte array that uses a synchronisation mechanism to allow threads to safely write in the byte array, then the other implementation must utilise the same kind of shared byte array that uses the same kind of synchronisation mechanism).

Most of the challenges met during the making of this part of the assignment came from the Python implementation, because in Python the possibility of true multi-threading (parallelism), as opposed to the traditional GIL-limited threading (concurrency only), is being introduced only in the experimental free-threaded build of Python 3.13; this means that the base and constructs for parallelisation on Python are not as streamlined and optimised as the ones present in C++. This means greater overhead in thread management and in memory access management, in addition to making it more difficult to keep the Python implementation coherent with the C++ implementation.

Both implementations proceed as follows:

- load the passwords from the files *rockyou.txt* and *parole_uniche.txt*.
- populate the dictionary using the *add* method.
- checks if the passwords of *parole_uniche.txt* are present inside the dictionary, using the *contains* method.

3.3 Performance Attributes

Firstly, we'll need to address the problem of false positives. In a Bloom filter, there cannot be false negatives due to the property of the Hash functions, but there can be false positives; this is caused by the fact that processing a password and, with the indices obtained, changing the bits indicated by the indices to 1s, loses any information about the password and, with the finite dimension of the bit array, the indices generated for a new password could coincide with bits already set to 1 by previously inserted passwords. The following formula describes the probability of having false positives:

$$p = \left(1 - \left[1 - \frac{1}{m} \right]^{kn} \right)^k$$

or approximated

$$p = \left(1 - e^{-\frac{kn}{m}} \right)^k$$

- **m** is the number of bits in the byte array (dynamically calculated and measured in bits).
- **k** is the number of hash functions required for processing the passwords.
- **n** is the number of elements of the dictionary.

With this formula, we can calculate the probability of having false positives. Taking into account the fact that we are working on a dictionary with 14 million passwords, even a probability of 2% can result in roughly 280'000 false positives. To minimise this, we set our acceptable threshold below 1% and selected parameters to achieve a target probability of 0.8%. In our test case we assigned **m** = 143443770, **k** = 6, **n** = 14344377, obtaining the following probability:

$$p = \left(1 - \left[1 - \frac{1}{143443770} \right]^{6 \cdot 14344377} \right)^6 = 0.00845$$

So, we predict that the expected number of false positives when querying 999997 passwords in our test case will be around 8,449, which is acceptable for the scope of this test. The predicted number of false positives will also be useful for checking the correctness of the Bloom Filters implementation.

The metrics used for the performance comparison, are:

- **Time to Complete (TTC)**: the execution time measured for both sequential and parallel implementations of the Bloom Filter operations.
- **Speedup**: indicates how much faster the parallel implementation is relative to the sequential implementation, measured with the formula: $speedup = \frac{TTC_{sequential}}{TTC_{parallel}}$
- **Efficiency**: quantifies how effectively a parallel algorithm utilizes the allocated hardware resources (processing cores or threads). It measures the proportion of processing power contributed by additional processors toward accelerating execution. It is obtained by the following formula $efficiency = \frac{speedup}{numberOfThreads}$

These metrics will be used internally to each version to compare the two implementations, as well as externally to compare the C++ version with the Python version.

3.4 C++ OpenMP Version - in-depth technical analysis

We began by implementing an abstract Bloom Filter that will be used as a base for the parallel and sequential implementations by encapsulating all the shared methods.

Before continuing with the methods description, it is important to note that the environment variables *OMP_PROC_BIND* and *OMP_PLACES* values have been changed to *OMP_PROC_BIND=splay* and *OMP_PLACES=threads*. This modifications basically tells the Operating System to schedule the threads on hardware threads rather than hardware real cores (because of Hyper-threading) and that thread distance between each other is one. This combination enforce thread pinning improving performances.

The abstract class has two public methods:

- *void add(const std::string& item)*, takes a password and its length as inputs, and passes them to the *_hash* method that returns two 64-bit hashes; these hashes are then passed to *_hash_single* method that, with the Double Hashing technique, returns **k** indices that are used by the method *_set_bit* to set the bits indicated by the indices to 0 or 1.

It also defines four protected methods:

- *std::array<uint64_t, 2> _hash(const uint8_t* data, std::size_t len)*, Takes a password and its size as input, then calculate the 128 bit hash of the password, using MurmurHash3, one time and returns the tuple (h1, h2) formed by the two 64bits parts of the hash through an *uint64_t* array with two positions.
- *size_t _hash_single(uint64_t h1, uint64_t h2, size_t hash_idx)*, this method takes as inputs two 64 bits hashes (the two parts of the 128 bit returned by *_hash*) and an index of *size_t* type; using the Double Hashing technique it returns the index of the position in the byte array where to put one.
- *void _set_bit(size_t bit_index)*, Takes an index as input, because of the bit-packing technique the method elaborates the index so that it points at the right bit in the right byte, and then set the bit to 1.
- *bool _get_bit(size_t bit_index)*, Does the same operations as the method *_set_bit* but instead of setting to 1 the indicated bit, it returns **true** if the bit is equal to 1, otherwise it returns **false**.

```
void add(const std::string& item) {
    auto [h1, h2] = _hash(reinterpret_cast<const uint8_t*>(item.data()),
                           item.size());

    for (size_t j = 0; j < K_HASHES; ++j) {
        _set_bit(_hash_single(h1, h2, j));
    }
}

bool contains(const std::string& item) const {
    auto [h1, h2] = _hash(reinterpret_cast<const uint8_t*>(item.data()),
                           item.size());

    for (size_t j = 0; j < K_HASHES; ++j) {
        if (!_get_bit(_hash_single(h1, h2, j))) {
            return false;
        }
    }
    return true;
}

std::array<uint64_t, 2> _hash(const uint8_t* data, std::size_t len) const {
    std::array<uint64_t, 2> hashValue;
    MurmurHash3_x64_128(data, len, 0, hashValue.data());
    return hashValue;
}
```

```

size_t _hash_single(uint64_t h1, uint64_t h2, size_t hash_idx) const {
    return static_cast<size_t>(h1 + hash_idx * h2) % size;
}

void _set_bit(size_t bit_index) {
    bit_array[bit_index >> 3] |= static_cast<uint8_t>(1u << (bit_index & 7));
}

bool _get_bit(size_t bit_index) const {
    return (bit_array[bit_index >> 3] &
            static_cast<uint8_t>(1u << (bit_index & 7))) != 0;
}

```

Both the sequential and parallel implementations override the virtual methods defined in the abstract base class:

- *void add_from_file(const std::vector<std::string>& items)*
- *size_t contains_from_file(const std::vector<std::string>& items)*

These methods are used to insert or check large batches of passwords from a file (roughly 14 million passwords in *rockyou.txt* and around 1 million in *parole.uniche.txt*). In the parallel implementation, these methods are parallelized using *OpenMP pragma* directives. These two methods are the only feasible candidates for parallelization, because the nested for loops are small and run only a few times per password. Parallelizing these loops would only introduce severe context-switching and thread management overhead while parallelizing the external for loops avoids re-initializing threads on every cycle.

```

void add_from_file(const std::vector<std::string>& items) override {
#pragma omp parallel num_threads(NUMBER_OF_THREADS)
#pragma omp for schedule(dynamic, 1024)
    for (size_t i = 0; i < items.size(); ++i) {
        auto [h1, h2] = _hash(reinterpret_cast<const uint8_t*>
                               (items[i].data()), items[i].size());
        for (size_t j = 0; j < K_HASHES; ++j) {
            size_t bit_index = _hash_single(h1, h2, j);

            size_t byte_index = bit_index >> 3;
            uint8_t mask = static_cast<uint8_t>(1u << (bit_index & 7));

#pragma omp atomic update
            bit_array[byte_index] = bit_array[byte_index] | mask;
        }
    }
}

```

As previously mentioned, the external for loop of the *add_from_file* method is executed, in this test case, around 14 million times, making this loop the primary objective for parallelization. So, the directive **#pragma omp parallel** is used to set the number of threads that will be utilized, using the clause *num_threads(NUMBER_OF_THREADS)* with value *omp_get_max_threads()* allows us to utilize all available CPU cores, so that different machines can avoid the overhead of thread management. We then use the *#pragma omp for* directive to divide the loop iterations among the threads. We set the scheduling policy to **dynamic** with a chunk size of **1024**, which dynamically balances the workload across threads to improve performances. We also tried to use the **collapse** clause to parallelize the inner for loop, but the use of this clause introduces overhead since for every external loop task, six additional tasks are created for the inner loop (so about 84000000 tasks in total) increasing thread switching and management overhead.

A race condition arises from the use of the bit-packing technique, because when simultaneous write operations are performed on the same byte will cause the last write operation to overwrite the previous operation and as a consequence, corrupts the byte. The solution that we choose is to use the *#pragma omp atomic update* directive to make sure that the write operation on the byte array cannot be interrupted and so ensuring their safety (the use of this directive introduces overhead, but it is a necessary evil).

```

size_t contains_from_file(const std::vector<std::string>& items) const override {
    size_t count = 0;
#pragma omp parallel num_threads(NUMBER_OF_THREADS)
#pragma omp for schedule(dynamic, 1024) reduction(+ : count)
    for (size_t i = 0; i < items.size(); ++i) {
        if (contains(items[i])) {
            count++;
        }
    }
    return count;
}

```

The *contains_from_file* method is lighter than *add_from_file*, and only has one for loop making it quite easy to parallelize. Its parallelization strategy is almost the same as in the *add_from_file* method, the only meaningful difference is that we used the clause **reduction(+ : count)** on the *#pragma omp for* directive, so that the thread-local results are safely accumulated into the count variable, correctly returning the total number of passwords found in the dictionary. Notice that, unlike the *add*, the *contains* operation does not perform *write* operations on it and thus does not need any atomic directive for loading bits from the bit array, and threads can read the same portions of bits from the same byte safely. This drastically simplifies the entire operation, avoiding any lock and obtaining a significant speed-up improvement, much more meaningful than the *add*; this will be noticeable later in performance analysis.

3.5 Python ThreadPooled version - in-depth technical analysis

To ensure a fair performance comparison between the C++ and Python implementations, the Python script was designed as a direct translation of the C++ implementation. This required adopting memory structures that closely match those used in C++, using equivalent parallelization strategies, and maintaining identical control flows across methods. The script follows the structure of its C++ counterpart in the following ways:

- it uses an abstract Bloom Filter class containing the implementations of the shared methods *add*, *contains*, *_set_bit*, *_get_bit*, *_hash_single* and *_hash*.
- the method *_hash* uses the same non-cryptographic hash function, MurmurHash3, and returns the same two 64 bit hashes; the *_hash_single* also uses Double Hashing to calculate the indices associated to the passwords.
- it uses a byte array of the same dimension, and the same bit-packing strategy to store the dictionary.
- It uses the exact same benchmark datasets (*rockyou.txt* to build the dictionary and *parole_uniche.txt* to evaluate false positives).
- It executes the same number of test iterations and evaluates the same performance metrics.

The parts of the Python script that are radically different from its C++ counterpart (obviously ignoring the differences intrinsic to the programming language itself) are in the methods *add_from_file* and *contains_from_file* in the parallel version of Bloom Filter. The most glaring difference is the *get_next_chunk* method:

```

def _get_next_chunk(self, total_items: int) -> tuple[int, int] | None:
    with self._sched_lock:
        start = self._next_index
        if start >= total_items:
            return None
        end = min(start + DYNAMIC_CHUNK_SIZE, total_items)
        self._next_index = end
        return start, end

```

The purpose of this method is to emulate the dynamic scheduling of the parallelized for loops in the C++ implementation. It basically takes the array of strings containing the passwords extracted from *rockyou.txt* and using the **DYNAMIC_CHUNK_SIZE** (in our test case it's equal to 1024) returns the

boundary indices of the chunk that a given thread must process.

The differences between *add_from_file* and *contains_from_file* with their C++ counterparts are caused by the different way in which Python manages threads and multithreading. Because of this both methods, rely on inner methods: methods (*worker()* for *add_from_file* and *process_chunk(chunk)* for *contains_from_file*). These methods are assigned and executed by the threads and contains the execution logic, while the parent method handles the creation and lifecycle management of the threads. Aside from this structural adaptation, the inner methods faithfully replicate the logic of their C++ counterparts.

```
def add_from_file(self, items: list[str]) -> None:
    self._next_index = 0
    total_items = len(items)

    def worker():
        while True:
            chunk = self._get_next_chunk(total_items)
            if chunk is None:
                break
            start, end = chunk
            for i in range(start, end):
                item_bytes = items[i].encode("utf-8")
                h1, h2 = self._hash(item_bytes)
                for j in range(K_HASHES):
                    bit_index = self._hash_single(h1, h2, j)
                    byte_index = bit_index >> 3
                    mask = 1 << (bit_index & 7)
                    lock = self._stripe_locks[byte_index % NUM_LOCK_STRIPES]
                    with lock:
                        self.bit_array[
                            byte_index
                        ] |= mask

    threads = [threading.Thread(target=worker) for _ in range(NUMBER_OF_THREADS)]
    for t in threads:
        t.start()
    for t in threads:
        t.join()

def contains_from_file(self, items: list[str]) -> int:
    chunk_size = DYNAMIC_CHUNK_SIZE

    def process_chunk(chunk):
        count = 0
        for item in chunk:
            if self.contains(item):
                count += 1
        return count

    chunks = [items[i : i + chunk_size] for i in range(0, len(items), chunk_size)]
    with ThreadPool(processes=NUMBER_OF_THREADS) as pool:
        results = pool.map(process_chunk, chunks)
    return sum(results)
```

This was the most challenging part of the assignment, because, to guarantee consistency and comparability between the two implementations, we were constrained to mimic the C++ implementation. This forced us to use strategies, structures and techniques that are sub-optimal and not the best course of action in Python.

3.6 Results

3.6.1 C++ OpenMP results

We executed 15 test runs to gather a reliable dataset for basic statistical analysis. The benchmark results, using 8 threads, are presented below:

Table 2: Performance results per cycle

Cycle	ADD				CONTAINS				False Positives (Seq/Par)
	Seq. (s)	Par. (s)	Speedup	Efficiency	Seq. (s)	Par. (s)	Speedup	Efficiency	
1	1.788680	0.772376	2.31582×	28.9477%	0.104945	0.0247040	4.24810×	53.1013%	8433 / 8433
2	1.828220	0.764494	2.39142×	29.8927%	0.120338	0.0246315	4.88553×	61.0691%	8433 / 8433
3	1.762170	0.753788	2.33775×	29.2219%	0.105156	0.0236571	4.44500×	55.5625%	8433 / 8433
4	1.730280	0.764255	2.26401×	28.3002%	0.105068	0.0246816	4.25694×	53.2117%	8433 / 8433
5	1.773250	0.755139	2.34825×	29.3531%	0.105639	0.0236245	4.47159×	55.8949%	8433 / 8433
6	1.745380	0.771000	2.26378×	28.2973%	0.105677	0.0245740	4.30035×	53.7544%	8433 / 8433
7	1.761440	0.741030	2.37701×	29.7127%	0.105181	0.0229278	4.58747×	57.3433%	8433 / 8433
8	1.773890	0.762907	2.32518×	29.0647%	0.105466	0.0244988	4.30494×	53.8118%	8433 / 8433
9	1.763390	0.739214	2.38549×	29.8186%	0.105510	0.0228405	4.61943×	57.7429%	8433 / 8433
10	1.767900	0.771050	2.29285×	28.6606%	0.105307	0.0246768	4.26748×	53.3434%	8433 / 8433
11	1.769770	0.738790	2.39550×	29.9437%	0.105596	0.0228229	4.62675×	57.8344%	8433 / 8433
12	1.775550	0.740277	2.39850×	29.9812%	0.105371	0.0228464	4.61213×	57.6516%	8433 / 8433
13	1.812030	0.738605	2.45332×	30.6665%	0.115830	0.0228247	5.07476×	63.4345%	8433 / 8433
14	1.726390	0.765169	2.25623×	28.2028%	0.105579	0.0246019	4.29150×	53.6437%	8433 / 8433
15	1.745580	0.740468	2.35740×	29.4675%	0.105222	0.0228112	4.61275×	57.6594%	8433 / 8433

Table 3: Performances Average

Operation	Avg. Seq. Time (s)	Avg. Par. Time (s)	Avg. Speedup	Avg. Efficiency
ADD	1.768260	0.754571	2.34417×	29.3021%
CONTAINS	0.107059	0.0237816	4.50698×	56.3373%

As expected, the average **TTC** (Time To Complete) of the parallel *add* operation is significantly lower than the **TTC** of the sequential operation. However, since the two **TTC** are relatively small values, the resulting **speedup** and **efficiency** are rather low, especially the **efficiency**. The reason for this sub-optimal efficiency is caused by the `#pragma omp atomic update` directive, which enforces thread synchronization and degrades overall performance; but this is still a noticeable improvement over the sequential operation. Since *contains* doesn't have any kind of synchronization among threads we can clearly see a sizable improvement in performance, having both **speedup** and **efficiency** at an acceptable level. We can also observe that our theoretical prediction regarding false positives holds true.

3.6.2 Python Threads results

Similarly to its C++ counterpart, the Python implementation was evaluated across 15 test runs using the same input datasets (*rockyou.txt* to populate the dictionary and *parole_uniche.txt* to measure the false positive rate). The results are summarized below:

Table 4: Performance results per cycle

Cycle	ADD				CONTAINS				False Positives (Seq/Par)
	Seq. (s)	Par. (s)	Speedup	Efficiency	Seq. (s)	Par. (s)	Speedup	Efficiency	
1	84.850860	90.787828	0.93×	11.68%	2.698682	1.393695	1.94×	24.20%	8539 / 8539
2	84.873115	92.670817	0.92×	11.45%	2.676591	1.157674	2.31×	28.90%	8539 / 8539
3	85.201686	75.509156	1.13×	14.10%	2.657308	1.071891	2.48×	30.99%	8539 / 8539
4	84.833558	72.810213	1.17×	14.56%	2.662760	1.038046	2.57×	32.06%	8539 / 8539
5	84.503808	72.435889	1.17×	14.58%	2.667830	1.078548	2.47×	30.92%	8539 / 8539
6	85.490919	91.068162	0.94×	11.73%	2.689181	1.186147	2.27×	28.34%	8539 / 8539
7	84.756644	98.812774	0.86×	10.72%	2.660590	1.048506	2.54×	31.72%	8539 / 8539
8	85.274003	90.334019	0.94×	11.80%	2.680425	1.074056	2.50×	31.20%	8539 / 8539
9	85.031182	85.598837	0.99×	12.42%	2.650066	1.061549	2.50×	31.21%	8539 / 8539
10	85.119402	73.845148	1.15×	14.41%	2.668620	1.124635	2.37×	29.66%	8539 / 8539
11	84.829701	71.654528	1.18×	14.80%	2.661025	0.986704	2.70×	33.71%	8539 / 8539
12	84.978749	72.504815	1.17×	14.65%	2.657608	1.044585	2.54×	31.80%	8539 / 8539
13	85.251395	90.167100	0.95×	11.82%	2.678232	1.150026	2.33×	29.11%	8539 / 8539
14	85.026555	88.767542	0.96×	11.97%	2.674827	1.132737	2.36×	29.52%	8539 / 8539
15	84.867235	90.416265	0.94×	11.73%	2.660707	0.960535	2.77×	34.63%	8539 / 8539

Table 5: Performances Average

Operation	Avg. Seq. Time (s)	Avg. Par. Time (s)	Avg. Speedup	Avg. Efficiency
ADD	84.992587	83.825540	1.03×	12.83%
CONTAINS	2.669630	1.100622	2.44×	30.53%

We would have expected, from the beginning, that the performances of the Python version would have been worse than its C++ counterpart. And we were right, by itself the **TTC** of the sequential versions of the operations 80 times slower than their C++ counterparts, but the most interesting find is that the parallel version of the operations is much slower than the sequential version. As said before, this massive degradation in performance is caused by the use of locks in the methods *get_next_chunk* and *add_from_file*, then adding the overhead of the Python language and the overhead of thread management, it becomes clear to see why the performances are the way they are. To improve the performances it would be needed a complete redesign of the structures, strategies and methods used, but this would mean departing from the C++ implementation making the two implementations no longer directly comparable. We still tried, to optimize the script, but, despite an improvement in the performances, the **TTCs**, speedups and efficiencies remained tens of times grater than the C++ implementation. We can assume that Python is not a suitable language for CPU-bound multithreading compared to C++. The prediction on the number of false positives still holds in this implementation.